## Лабораторная работа № 8
## Получение и отображение данных

**Цель работы**

Изучить вопросы получения данных из БД оператором запроса, обработки полученного набора данных. Изучить отображение полученных данных в графическом в табличном интерфейсе.

### 1. Теоретические сведения

После того как установлено соединение с БД, оно используется для выполнения SQL-запросов на получение и изменения данных.

В JDBC для посылки SQL-запросов используется объект класса соединения с БД `Connection` и объект класса для формирования операторов запроса к БД `Statement`.

Для создания объекта Statement, включающего текст оператора запроса, используется метод createStatement класса соединения:

```
Statement stmt = conn.createStatement();
```

Для выполнения запроса к БД используется метод `executeQuery` объекта класса `Statement`. Полученный результат возвращается в виде набора данных (объекта класса `ResultSet`):

```
ResultSet rset = stmt.executeQuery ("SELECT * FROM Sprod");
```

который в дальнейшем можно использовать для обработки или отображения

Объект класса `ResultSet` поддерживает курсор - указатель на текущую строку набора данных. При получении набора данных курсор позиционируется перед первой строкой набора. Для изменения положения курсора используется метод набора данных `next()`, который перемещает курсор на следующую запись. Если записи исчерпаны, то метод возвращает значение `false`. Просмотр строк полученных данных выполняется в цикле до тех пор, пока не будет достигнут конец набора данных. Для получения значения колонки текущей строки используется метод набора данных

```
rset.getXXX(n),
```

где XXX – тип Java, например, getString(),
n – номер колонки. Нумерация колонок начинается с 1.

Все операции будем выполнять на примере БД учета продажи продукции. БД разработана в программном средстве моделирования данных *SQL Power Architect*. Была разработана логическая (рис.1) и физическая (рис.2) модели данных.

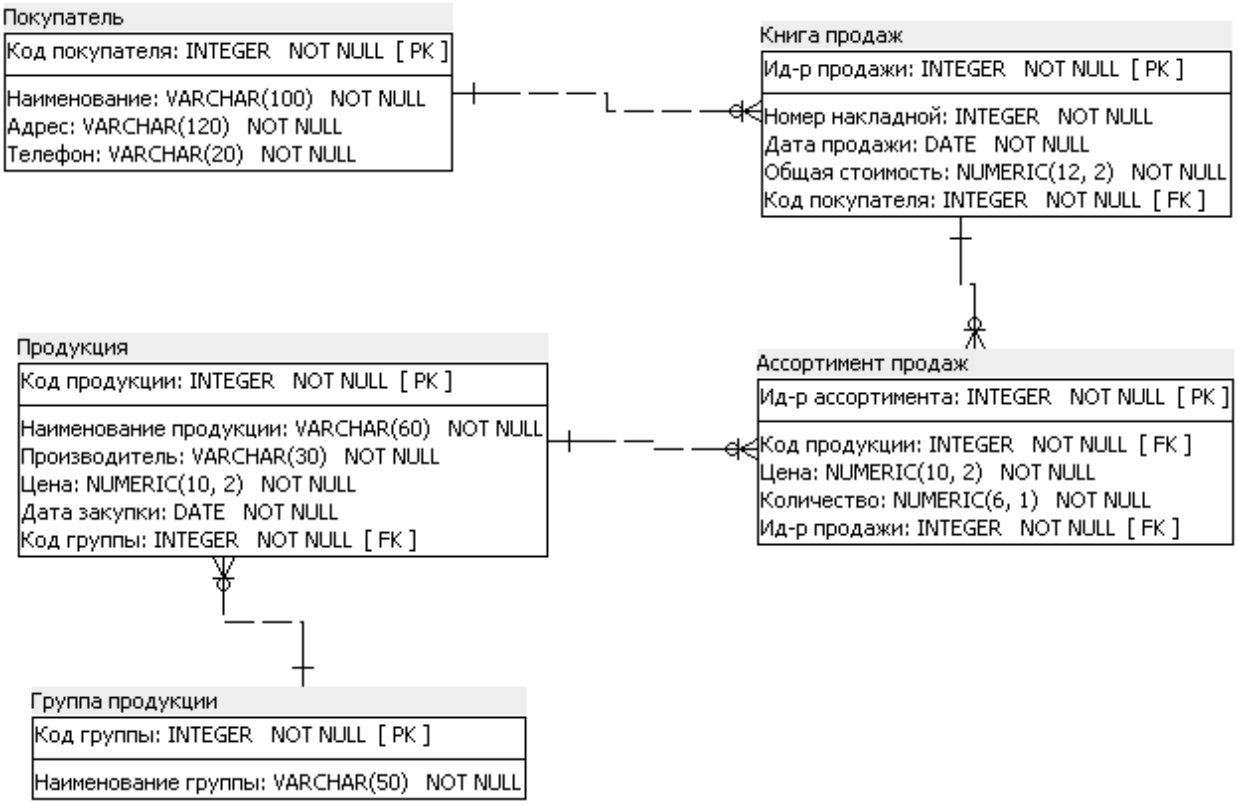



***Рис.1 Логическая модель данных***

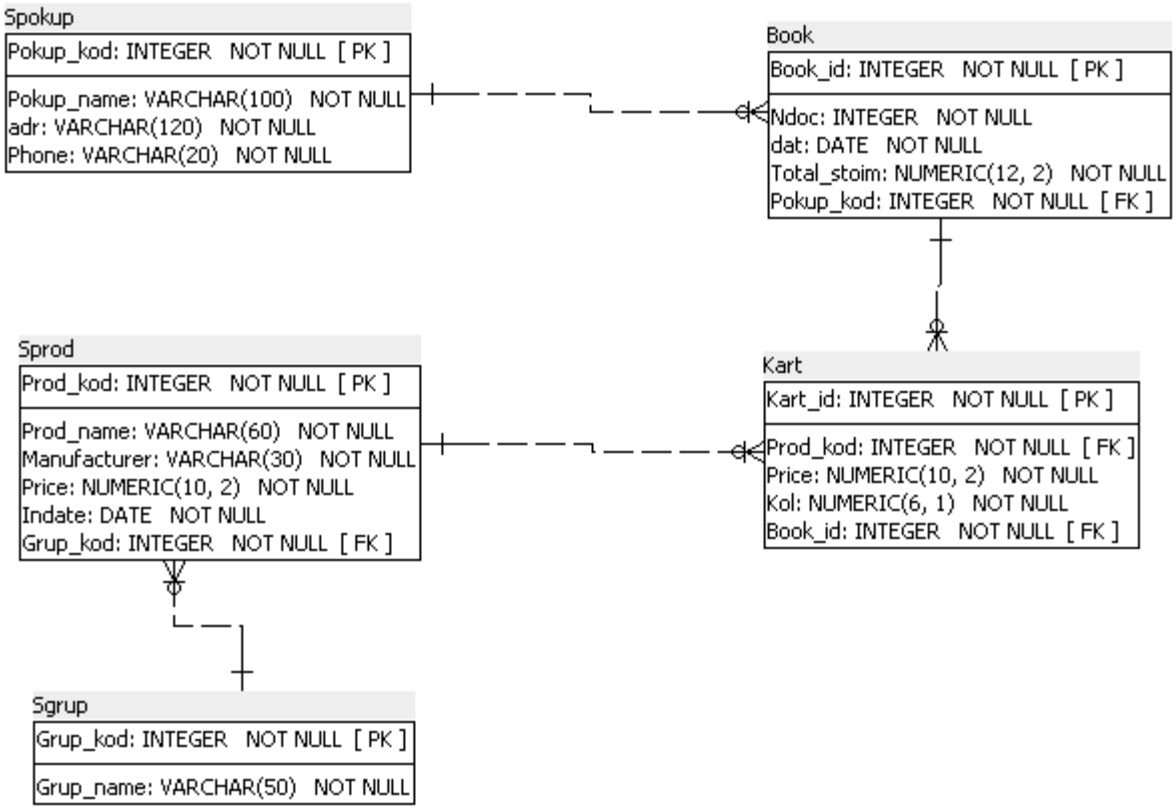



***Рис.2 Физическая модель данных***

Пока в программном приложении не созданы средства редактирования данных, введем данные вручную, чтобы можно было тестировать

приложение в данной лабораторной работе. Для этого используя утилиту PgAdmin, выполним операторы добавления данных в 2 таблицы: Группа, Продукция:

```
INSERT INTO prod.sgrup(grup_kod, grup_name)
    VALUES (1, 'Память');
INSERT INTO prod.sprod(
    prod_kod, prod_name, manufacturer, price, indate, grup_kod)
    VALUES (1,'Память DDR3','Kingston',2500,'2020-02-15',1);
```

Реализацию функций получения и отображения данных рассмотрим на примере только одной таблицы Продукция (*Sprod*). Для остальных таблиц все действия, описанные в данной работе, можно повторить с определенными изменениями. Изменим приложение, созданное в предыдущей лабораторной работе, добавив функции извлечения данных из БД и отображения их в интерфейсе пользователя.

В предыдущей работе рассматривались вопросы начала разработки приложения и установки соединения приложения с БД. В данной работе изменим класс основного окна приложения ***MainWindowControl***, созданного в предыдущей лабораторной работе, и добавим новые классы.

Для отображения данных из таблицы БД Продукция выполним следующие шаги:

1) Создадим классы, объектами которых будут Группа и Продукция

2) В классе Менеджера работы с БД **DBManager** добавим метод получения наборов данных из БД и формирования коллекции объектов Продукция из полученных данных.

3) Создадим класс окна для отображения данных в табличном интерфейсе.

4) Изменим класс основного окна **MainWindowControl**. Добавим меню с пунктами вызова дополнительных форм приложения

Рассмотрим подробно эти шаги:

### 1) Создание классов данных объектов Группа и Продукт

В JavaFX для всех полей класса данных (модели) предпочтительно использовать объекты **Properties** вместо **Fields**. Класс **Property** позволяет получать автоматические уведомления при любых изменениях полей класса. Это позволяет поддерживать синхронность между данными (Model) и представлением (View).

Создадим в пакете **com.data** классы **Grup** (Группа) **Product** (Продукт).

Класс Grup

```
package com.data;
public class Grup {
    //  поля класса
    private ObjectProperty<BigDecimal> grupKod;
    private StringProperty grupName;
```

```
	// Конструктор с инициализацией полей
	public Grup(BigDecimal grupKod, String grupName) {
		this.grupKod =
			new SimpleObjectProperty<BigDecimal>(grupKod);
		this.grupName = new SimpleStringProperty(grupName);
	}
	 // Конструктор без параметров
	public Grup() {
		this(null, null)
	}
	//   get/set для поля grupKod
	//   Для остальных полей методы get/set - формируются автоматически
	public final ObjectProperty<BigDecimal> grupKodProperty() {
		return grupKod;
	}
	public final BigDecimal getGrupKod() {
		return grupKodProperty().get();
	}
	public final void setGrupKod(final BigDecimal grupKod) {
		this.grupKod.set(grupKod);
	}
	@Override
	// Метод возвращает общую информацию о группе
	public String toString(){
	    return getGrupName();
	}
}
```

Класс `Product`

```
package com.data;
public class Product {
	//  поля класса
	private ObjectProperty<BigDecimal> prodKod; // Код
	private StringProperty prodName;            // Наименование
	private StringProperty manufacturer;        // Производитель
	private ObjectProperty<BigDecimal> price;   // Цена
	private ObjectProperty<LocalDate> indate;   // Дата_закупки
	private ObjectProperty<Grup> grup;          // Группа
	// Конструктор с инициализацией полей
	public Product(BigDecimal prodKod, String prodName,
			String manufacturer, BigDecimal price,
			LocalDate indate, Grup grup) {
		this.prodKod = new
			SimpleObjectProperty<BigDecimal>(prodKod);
		this.prodName = new SimpleStringProperty(prodName);
		this.manufacturer = new SimpleStringProperty(manufacturer);
		this.price = new SimpleObjectProperty<BigDecimal>(price);
		this.indate = new SimpleObjectProperty<LocalDate>(indate);
		this.grup = new SimpleObjectProperty<Grup>(grup);
	}
	 // Конструктор без параметров
	public Product() {
		this(null, null, null, null, null, null);
	}
	//   get/set для поля prodKod
	//   Для остальных полей get/set - формируются автоматически
	public final ObjectProperty<BigDecimal> prodKodProperty() {
		return this.prodKod;
```

```
        }
        public final BigDecimal getProdKod() {
              return this.prodKodProperty().get();
        }
        public final void setProdKod(final BigDecimal prodKod) {
              this.prodKodProperty().set(prodKod);
        }
        @Override
        // Метод возвращает общую информацию о продукте
        public String toString(){
            return getProdName();
        }
}
```

*Пояснения*

1. Для полей используем классы-обертки над обычными типами данных (реализуют интерфейс **`Property`** для произвольного класса **`Object`**): **`StringProperty`** для строк и **`ObjectProperty<тип>`** для других типов данных
2. Поле **`grup`** является классом *Группа*
3. Добавляем 2 конструктора: конструктор с инициализацией всех полей и конструктор без параметров, которой инициализирует все поля значениями null
4. Для автоматической генерации методов get/set необходимо выделить поля и выполнить пункт меню (рис. 3)

   **`Source – Generate JavaFX Getters and Setters`**

   Все методы являются финальными.

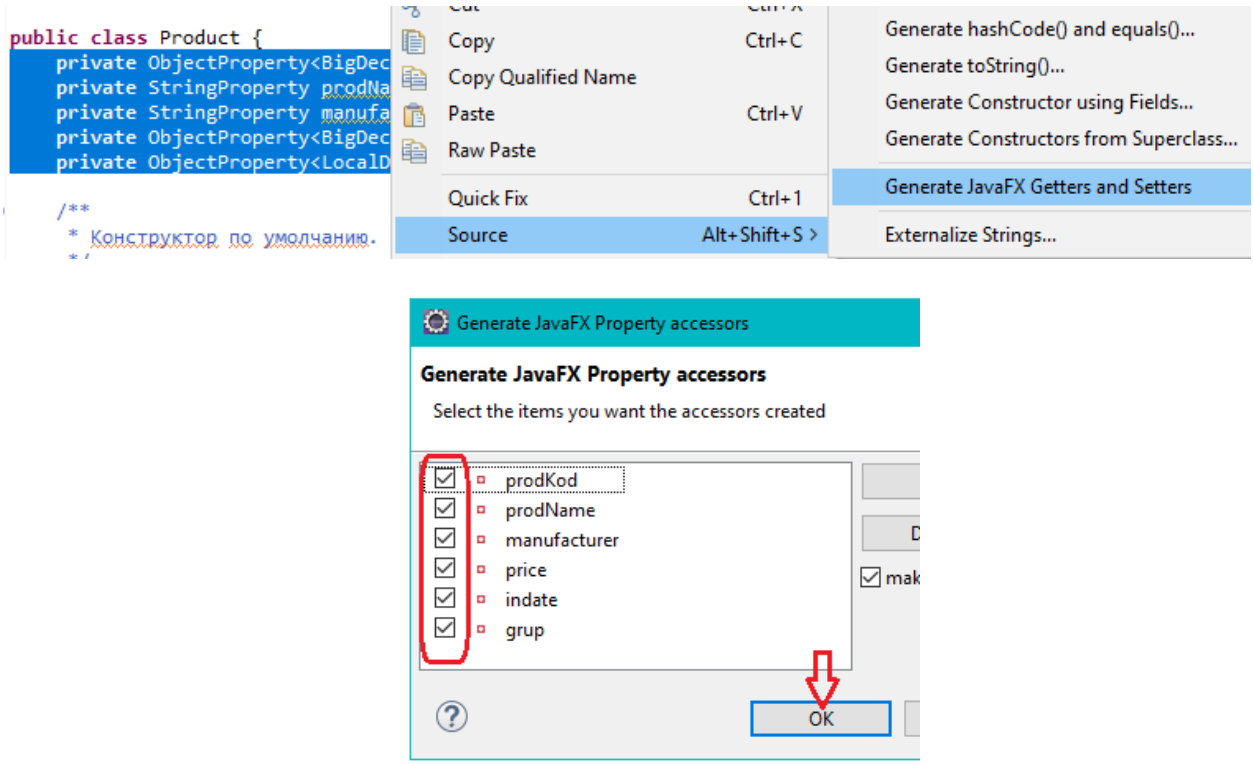



***Рис.3 Генерация для полей методов set и set***

## 2) Изменения в классе взаимодействия с БД **DBManager**

Добавим в класс **DBManager** метод отмены транзакции в ситуации ошибки при выполнении запроса.

```
	private void RollBack() {
		try {
			conn.rollback();
		} catch (SQLException e) {
			e.printStackTrace();
		}
	}
```

Добавим метод выполнения запроса на получения данных из таблицы Sprod. Метод возвращает объект «наблюдаемого» списка **ObservableList**. Запрос выполняется не к таблице **Sprod**, а к представлению **SPROD_V**, т.к. необходимо также получить значение наименования группы.

```
public ObservableList<Product> loadProds() {
	ObservableList<Product> prods = FXCollections.observableArrayList();
	// Получение объекта соединения
	Connection con = this.getConnection();
	try {
		// Формирование запроса к БД
		Statement stmt = con.createStatement();
		// Выполнение запроса и получение набора данных
		ResultSet res = stmt.executeQuery(
			"SELECT prod_kod,prod_name,manufacturer,price,indate"
			+ ",grup_kod,grup_name FROM SPROD_V order by prod_kod");
		// В цикле просмотра набора данных формируем список
		while (res.next()) {
			Product pr = new Product();
			pr.setProdKod(res.getBigDecimal(1));
			pr.setProdName(res.getString(2));
			pr.setManufacturer(res.getString(3));
			pr.setPrice(res.getBigDecimal(4));
			if (res.getDate(5) != null)
				pr.setIndate(res.getDate(5).toLocalDate());
			else
				pr.setIndate(null);
			// Создание объекта Группа
			Grup gr = new Grup();
			gr.setGrupKod(res.getBigDecimal(6));
			gr.setGrupName(res.getString(7));
			pr.setGrup(gr);
			// Добавление объекта в коллекцию
			prods.add(pr);
		}
	} catch (SQLException e) {
		Dialogs.showDialog("Ошибка получения данных",
			e.getMessage(), AlertType.ERROR);
		RollBack();
	}
	return prods;
}
```

Скрипт создания представления

```
 CREATE OR REPLACE VIEW prod.sprod_v
```

```
        AS
        SELECT pr.prod_kod,
           pr.prod_name,
           pr.manufacturer,
           pr.price,
           pr.indate,
           pr.discont,
           pr.grup_kod,
           gr.grup_name
          FROM prod.sprod pr
           LEFT JOIN prod.sgrup gr ON pr.grup_kod = gr.grup_kod;
```

### 3) Создание окна для вывода данных продукции

Результат запроса к представлению БД выведем в пользовательском интерфейсе. Отображение результатов выполним в табличном компоненте *`TableView`* (рис.4).

Продукция

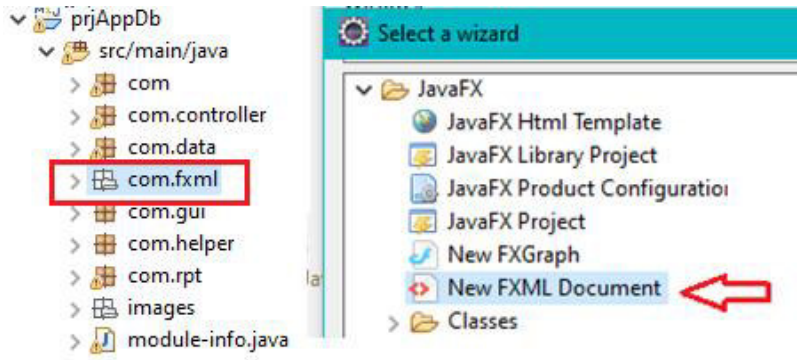
| ... | Наименование | Производитель | Цена | Дата закупки | Группа |
|---|---|---|---|---|---|
| 1 | Процессор i7 | Intel | 9000.00 | 17-10-2024 | Видеокарты |
| 4 | Видеокарта GW-450 | Radeon | 4000.00 | 21-08-2017 | Видеокарты |
| 5 | Видеокарта BB-600 | NVidia | 3400.00 | | Видеокарты |
| 6 | Оперативная память DDR4 4Гб | Kingston | 6000.00 | | Процессоры |
| 7 | SSD Samsung | Тот еще | 1502.00 | 14-11-2024 | Память |
| 9 | DDR3 | Samsung | 1200.00 | 13-11-2024 | Накопители |
| ... | Процессок Intel i5 | Intel | 5500.00 | 09-11-2023 | Процессоры |

***Рис.4 Таблица Продукция***

Окно реализуется двумя файлами: файлом **`fxml`** с описанием графического интерфейса и файлом класса контроллера.

Создадим в пакете **`com.fxml`** файл графического интерфейса **`ProductView.fxml`** (рис. 5)

**`File – New – Other – New FXML Document`**



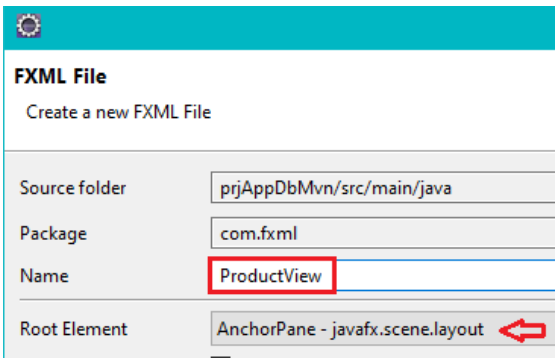



*Рис.5 Создание файла fxml*

Откроем файл **`ProductView.fxml`** в построителе GUI Scene Builder. На вкладке **`Hierarchy`** будет находиться один компонент панели **`AnchorPane`** (рис. 6).

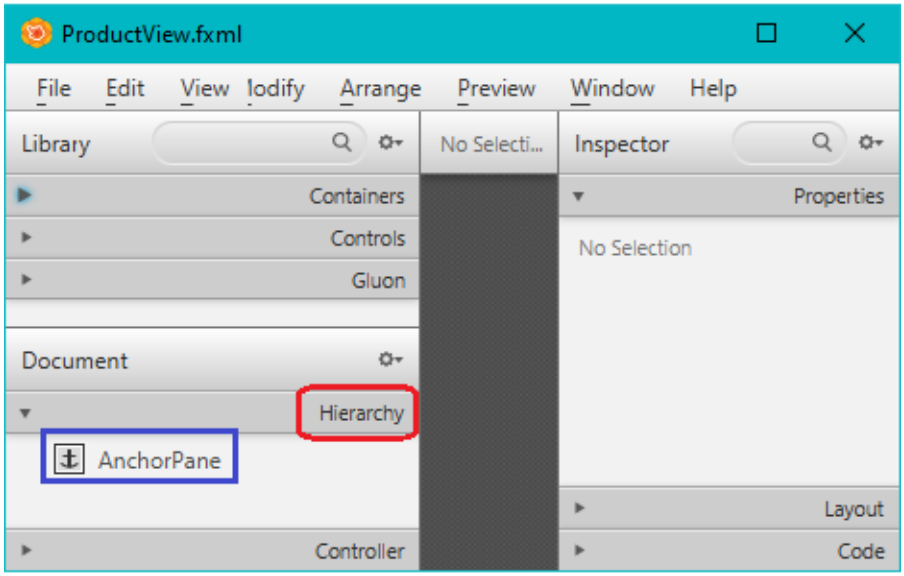



*Рис.6 Начальный вид окна `ProductView.fxml`*

В левой части окна на вкладке **`Hierarchy`** выберем компонент **`AnchorPane`**, и в правой части окна на вкладке **`Layout`**, установим значение свойств:

**`Pref Width = 800  Pref Height = 400`**

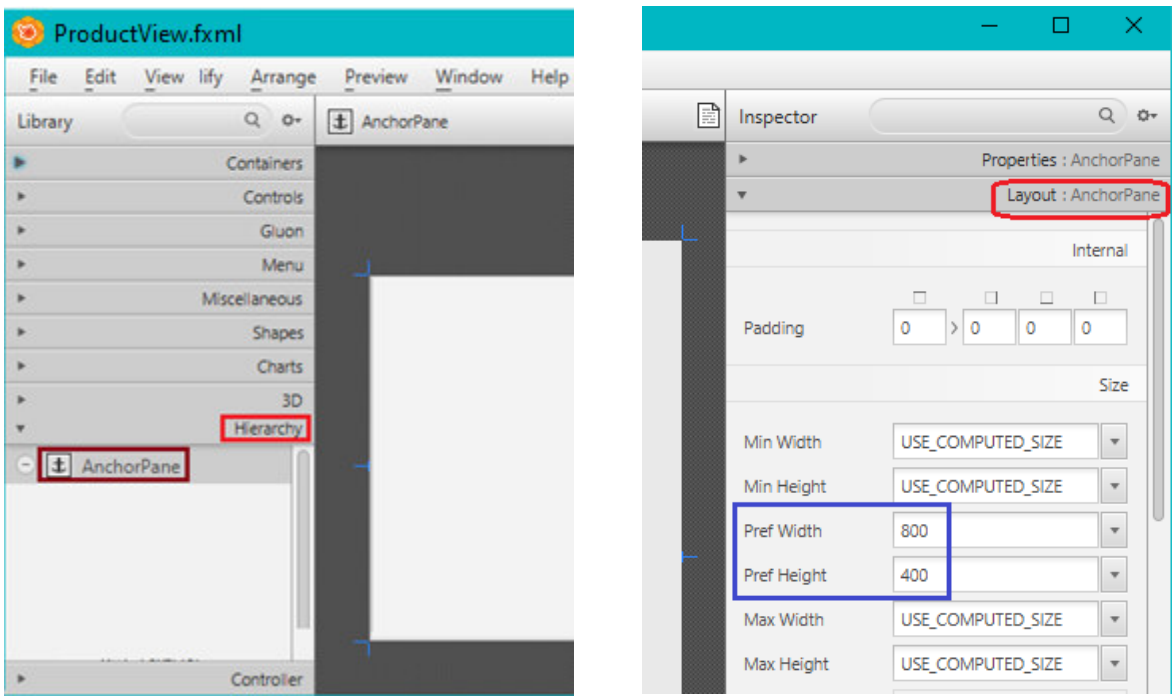
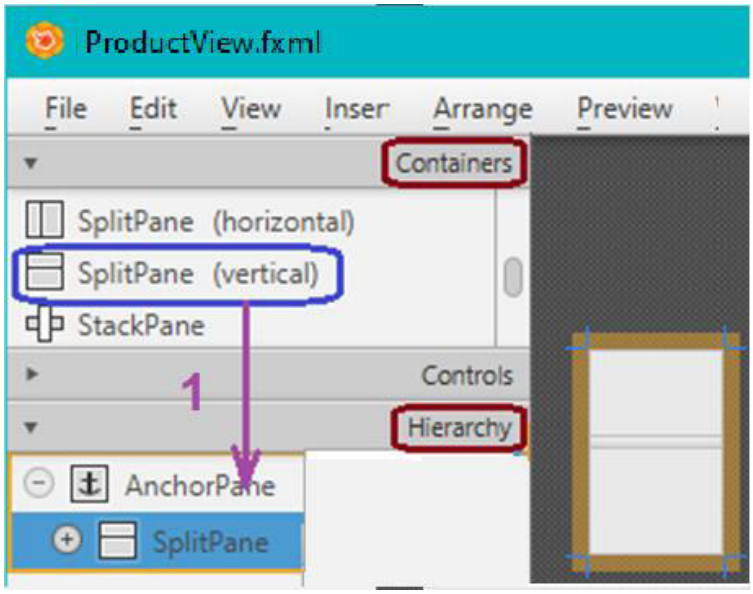



***Рис.7 Установка размеров окна***

В левой части окна перетащим с вкладки **`Containers`** компонент **`SplitPane(vertical)`** в компонент **`AnchorPane`** на вкладке **`Hierarchy`**.

В контекстном меню нового компонента выберем (рис. 8)

**`Fit to Parent`**



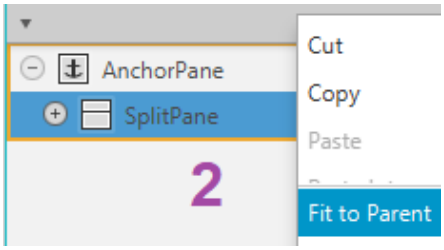




*Рис.8 Добавление панели* ***SplitPane*** *и привязка к границам окна*

Другой способ привязки нового контейнера к границам родительского контейнера – выставить поля на вкладке **Layout** (рис.9).

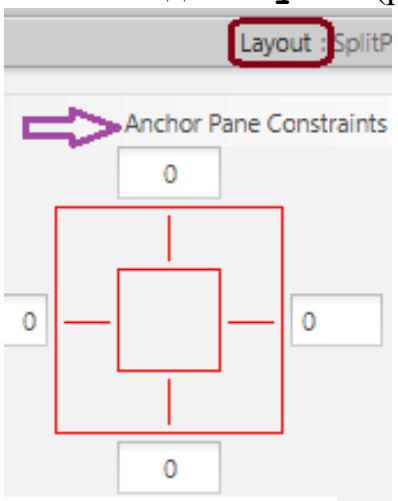



*Рис.9 Установка отступов от границ родительского контейнера* ***SplitPane*** *и привязка к границам окна*

Захватим и передвинем разделительную полосу между панелями **SplitPane** вниз. Тем самым основную часть окна будет занимать верхняя панель.

Перетащим с вкладки **Controls** компонент **TableView** в верхнюю часть **SplitPane** (рис. 10).

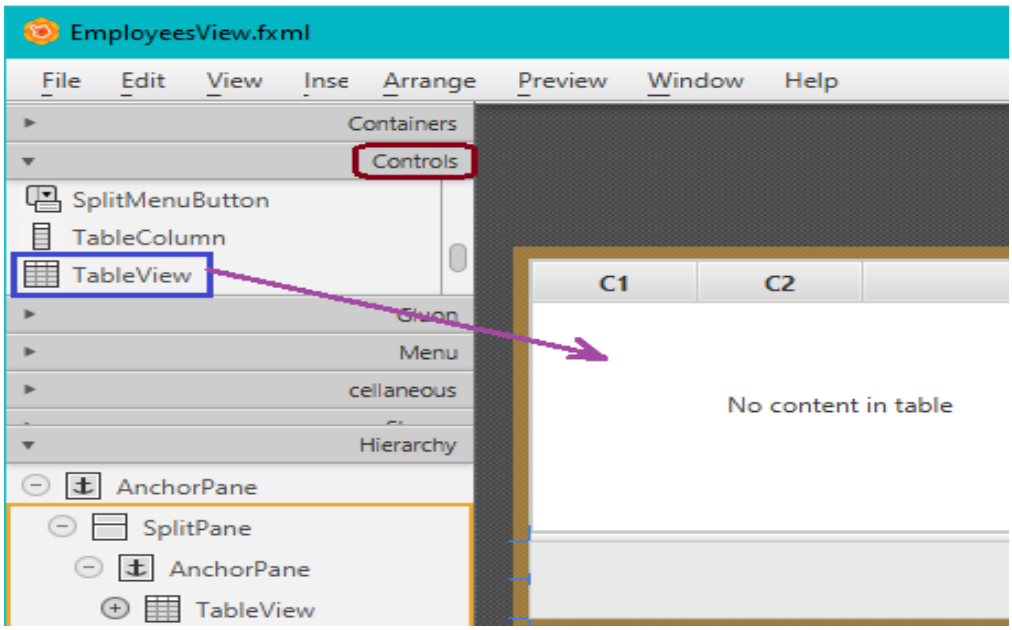



*Рис.10 Добавление компонента* ***TableView***

Также растянем **TableView** на все пространство панели **SplitPane**, выполнив **Fit to Parent**.

Добавим новые колонки в **TableView**. Выделим последнюю колонку и в контекстном меню выберем **Duplicate** (рис.11). Повторим операцию нужное количество раз, чтобы разместились все колонки таблицы **Product**.

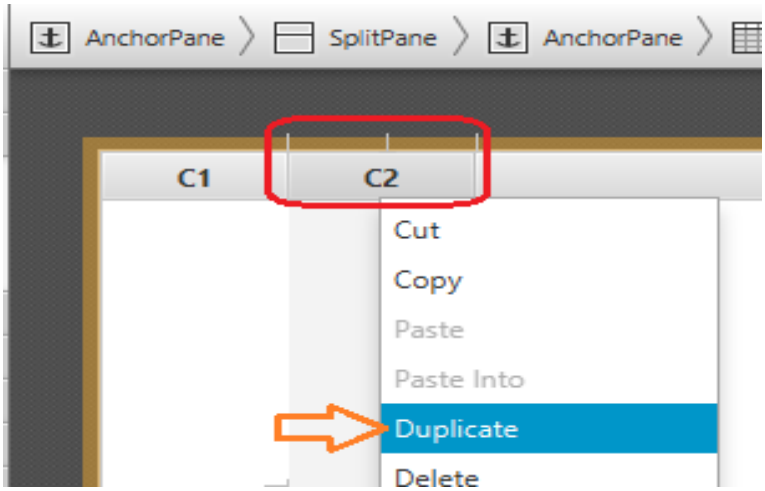



*Рис.11 Добавление колонок в компонент* ***TableView***

Для компонента **TableView** на вкладке **Properties** изменим значение **Column Resize Policy** на **constrained-resize** (рис.12). Это гарантирует, что колонки таблицы всегда будут занимать всё доступное пространство.

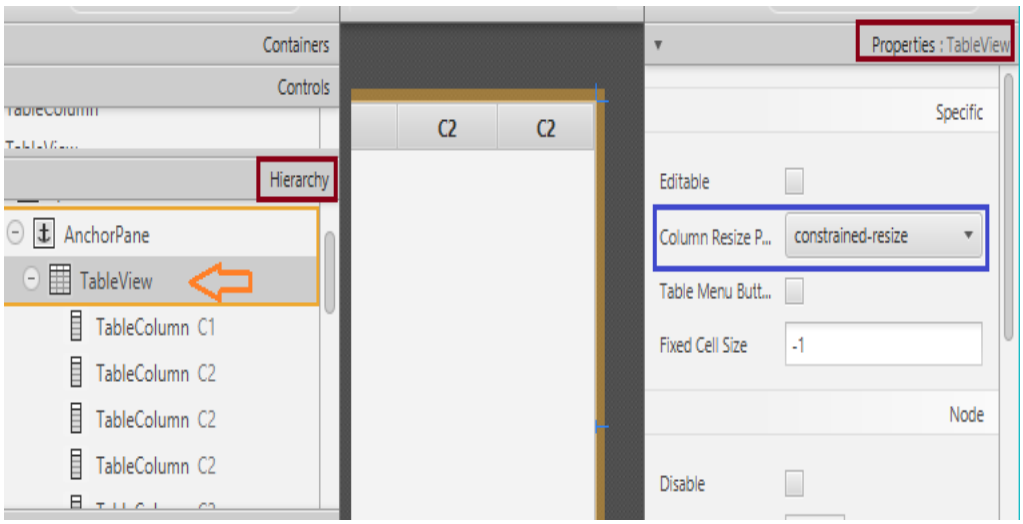



*Рис.12 Настройка растягивания колонок по ширине таблицы*

Изменим заголовки колонок. Выделим колонку и на вкладке **`Properties`** изменим свойство **`Text`** (рис. 13).

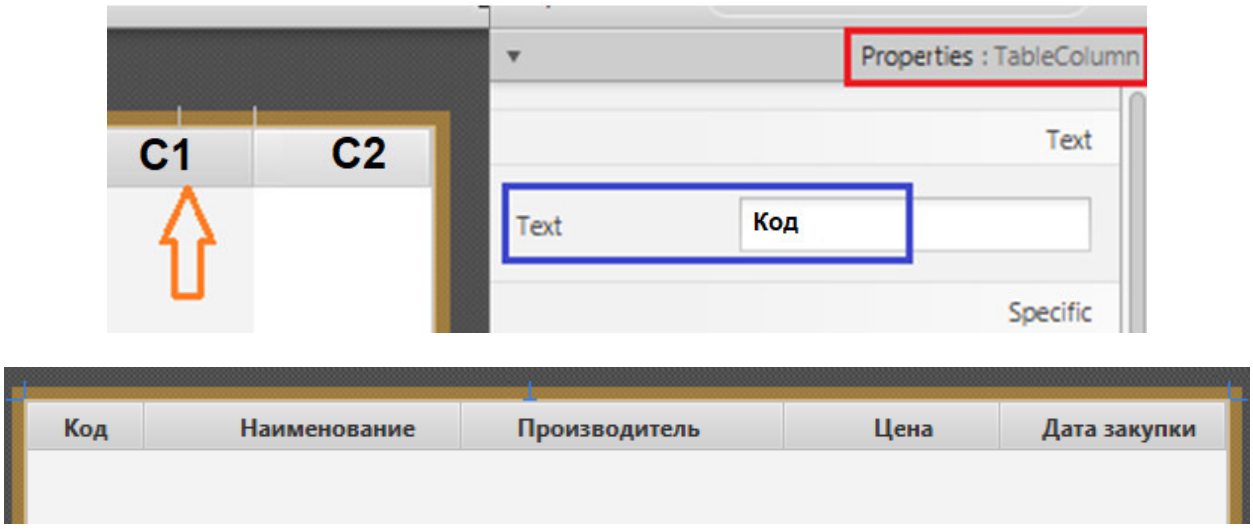



*Рис.13 Изменение названий колонок таблицы*

Нижнюю часть панели оставим пока пустой. В нее добавим кнопки для редактирования таблицы в следующей работе.

Для просмотра результата сформированного интерфейса выведем его в режиме просмотра, выбрав пункт меню **`Preview`** (рис. 14).

Сохраним сформированный интерфейс, выбрав пункт меню

**`File-Save`**

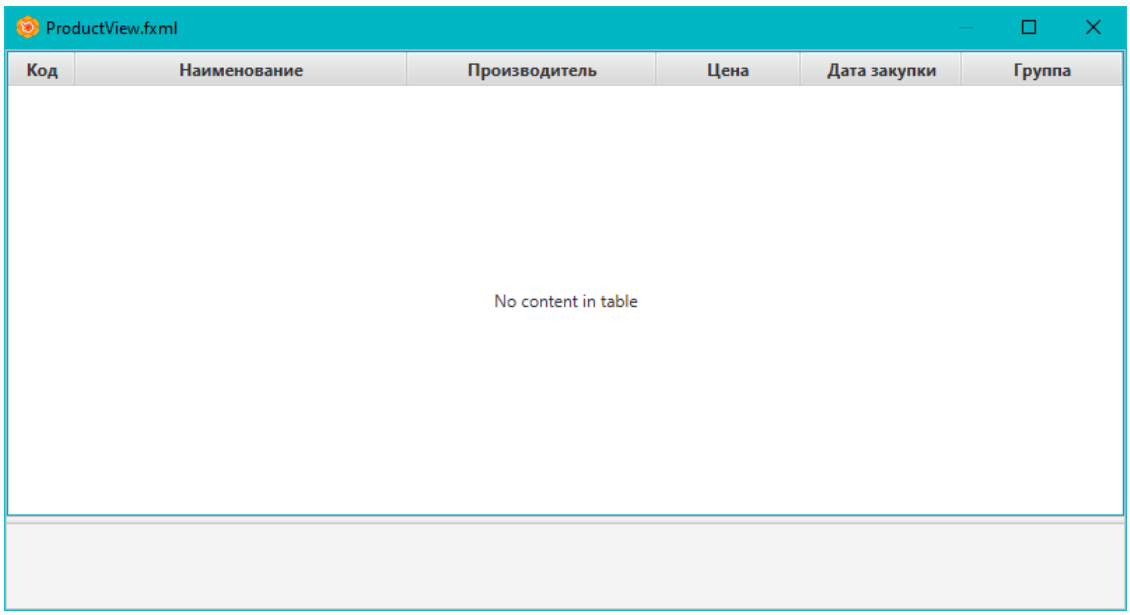



***Рис.14 Интерфейс окна***

Создадим в пакете **`com.controllers`** класс контроллера для вывода окна **`ProductViewController`** . Класс должен реализовывать интерфейса **`Initializable`** (рис. 15).

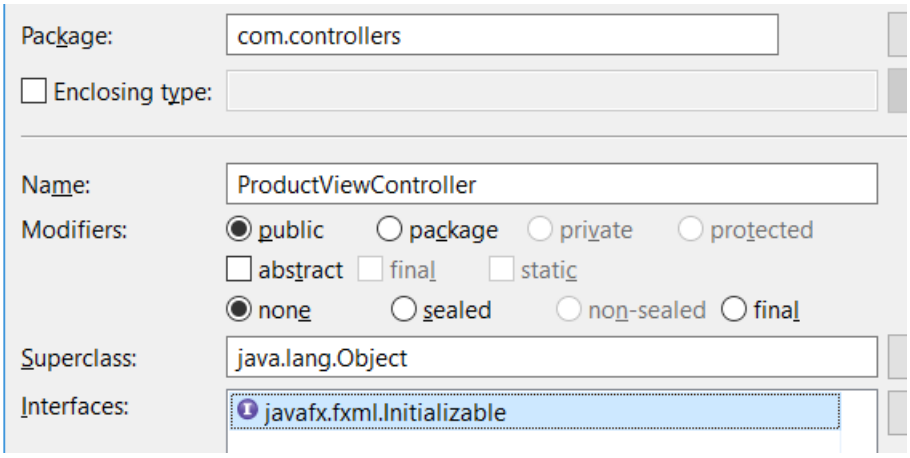



***Рис.15 Создание класса контроллера***

Программный код класса контроллера

```
public class ProductViewController implements Initializable {
      // Поля класса
      @FXML
      private TableView<Product> prodTable;
      @FXML
      private TableColumn<Product, BigDecimal> prodKodColumn;
      @FXML
      private TableColumn<Product, String> prodNameColumn;
      @FXML
      private TableColumn<Product, String> manufacturerColumn;
      @FXML
```

```
    private TableColumn<Product, BigDecimal> priceColumn;
    @FXML
    private TableColumn<Product, LocalDate> indateColumn;
    @FXML
    private TableColumn<Product, String> grNameColumn;
    // Набор «наблюдаемых» данных для таблицы
    private ObservableList<Product> prodData;

    private DBManager manager;
    private Stage dialogStage;

    // Перекрываемый метод - оставляем пустым
    @Override
    public void initialize(URL arg0, ResourceBundle arg1) {
    }
    // Собственный метод инициализации
    public void initialize(Stage dialogStage, DBManager manager) {
          // Установка сцены окна
          this.dialogStage = dialogStage;
          // Установка менеджера
          this.manager = manager;
          // Загрузка данных
          prodData = manager.loadProds();

          // Добавление в таблицу данных из наблюдаемого списка
          prodTable.setItems(prodData);

          // Инициализация таблицы продуктов
          prodKodColumn.setCellValueFactory(cellData ->
                cellData.getValue().prodKodProperty());
          prodNameColumn.setCellValueFactory(cellData ->
                cellData.getValue().prodNameProperty());
          manufacturerColumn.setCellValueFactory(cellData ->
                cellData.getValue().manufacturerProperty());
          priceColumn.setCellValueFactory(cellData ->
                cellData.getValue().priceProperty());
          indateColumn.setCellValueFactory(cellData ->
                cellData.getValue().indateProperty());
          grNameColumn.setCellValueFactory(
                      new Callback<CellDataFeatures<Product,String>,
                      ObservableValue<String>>(){
            @Override
            public ObservableValue<String> call(CellDataFeatures<Product,
                              String> param) {
                return new SimpleStringProperty(param.getValue().
                      getGrup().getGrupName());
            }
        });
    }
}
```

Пояснения

1. Полями класса являются компонент таблицы **`TableView`**, колонки таблицы **`TableColumn`**, к которым FXML-файлу потребуется доступ. Для этого пометим их аннотацией @FXML
2. Набор данных для таблицы – «наблюдаемый» список **`ObservableList<Product>`**, который выполняет функцию модели данных. Список заполняется данными, получаемым в результате запроса

к БД, который выполняется в методе **loadProds** в классе **DBManager**. Список передается конструктору табличного компонента.

3. Таблица, колонки не создаются в классе контроллера (для них здесь не вызывается конструктор). Инициализация выполняется неявно, когда происходит загрузка файла FXML. Это будет выполняться в классе главного окна в методе вызова данного окна.
4. Сопоставление данных поля набора и колонок таблицы выполняется в операторах
```
колонка.setCellValueFactory(cellData ->
            cellData.getValue().поле-свойство());
```
5. Особым образом получается колонка *Наименование группы*. Она извлекается из объекта-класса *Группа* по колонке внешнего ключа *Код группы*

Связываем класс контроллера с файлом описания интерфейса. Для этого откроем файл **ProductView.fxml** в **Scene Builder.**

Вначале укажем класс контроллера. На вкладке **Controller** в поле **Controller class** введем имя класса (с указанием пути класса в проекте) **com.controllers.ProductViewController**.

Далее настроим связь для таблицы и ее колонок (рис. 16). Выберем компонент **TableView** на вкладке **Hierarchy**. Перейдем на вкладку **Code** и установим в поле **fx:id** значение **prodTable** (имя должно совпадать с именем в контроллере).

***Рис.16 Задание имени табличного компонента***

Зададим имена колонок таблицы (рис.17). Выделим первую колонку **Код** и установим в поле **fx:id** значение **prodKodColumn** (имя должно совпадать с именем в контроллере). Выполним то же самое для других колонок таблицы.

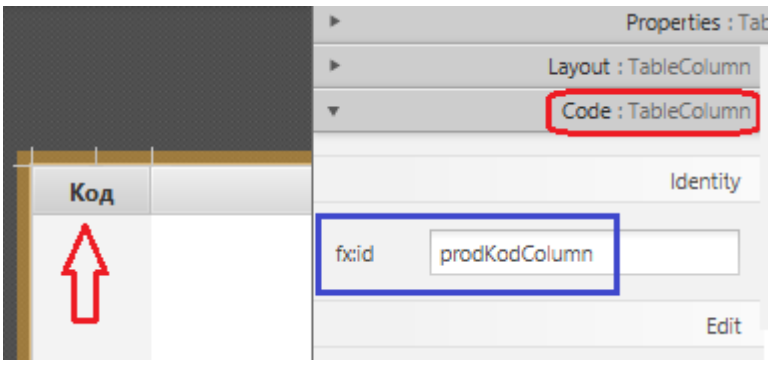



*Рис.17 Задание имени колонки*

Сохраним файл FXML (**Save**), а в среде разработки обновим весь проект (**F5**). Это необходимо для того, чтобы приложение «увидело» изменения, выполненные в Scene Builder.

**4) Изменим класс основного окна приложения**.

В класс **MainWindowController** добавим метод вызова окна просмотра данных из таблицы БД Продукция. Метод пометим аннотацией @FXML

```
@FXML
private void handleProduct() {
      try {
            // Загружаем fxml-файл и создаём новую сцену
            // для диалогового окна.
            FXMLLoader loader = new FXMLLoader();
            loader.setLocation(
                  MainApp.class.getResource("fxml/ProductView.fxml"));
            AnchorPane page = (AnchorPane) loader.load();
            // Создаём диалоговое окно Stage
            Stage viewStage = new Stage();
            // Задаем заголовок окна
            viewStage.setTitle("Продукция");
            // Определяем окно как модальное
            viewStage.initModality(Modality.WINDOW_MODAL);
            // Определяем текущее окно владельцем нового окно
            viewStage.initOwner(dialogStage);
            // Создаём сцену нового окна
            Scene scene = new Scene(page);
            viewStage.setScene(scene);
            // Загрузка иконки
            InputStream iconStream =
                  getClass().getResourceAsStream("/images/prod16.png");
            if (iconStream == null)
                  System.out.println("Файл не найден");
            else {
                  Image image = new Image(iconStream);
                  viewStage.getIcons().add(image);
            }
            // Загружаем контроллер
            ProductViewController controller = loader.getController();
            // Инициализируем компоненты окна
            controller.initialize(viewStage, manager);
            // Отображаем диалоговое окно с ожиданием его закрытия
            viewStage.showAndWait();
```

```
        } catch (IOException e) {
            e.printStackTrace();
        }
}
```

В файле **`MainWindow.fxml`** добавим в главное меню в подменю *Справочники* пункт (объект **`ItemMenu`**) *Продукция* (рис. 18), и назначим ему обработчик – метод **`handleProduct`** (рис. 19).

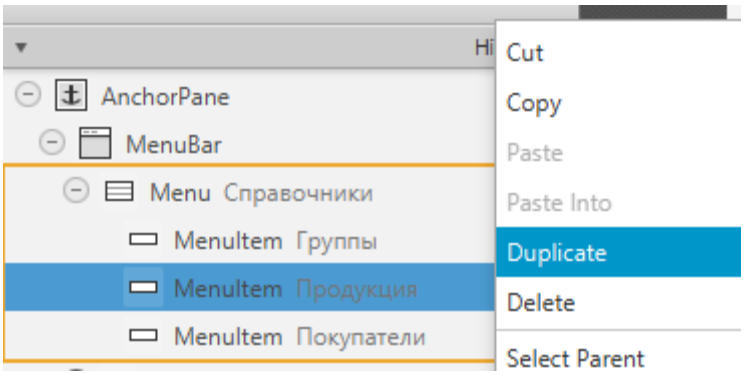



***Рис.18 Добавление пункта меню Продукция***

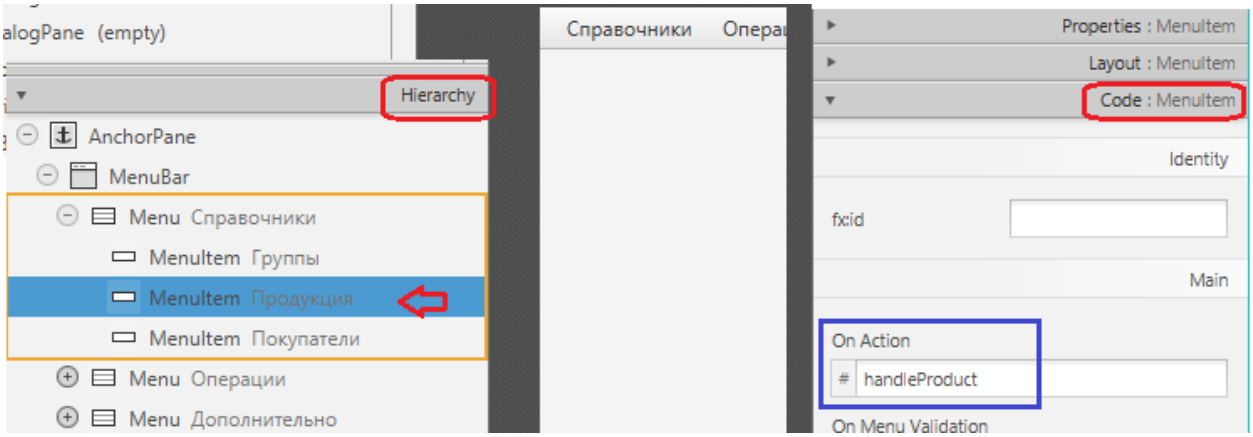



***Рис.19 Назначение пункту меню обработчика***

Запустим программу и выберем пункт меню *Продукция* (рис.20). Будет выведено окно с представлением данных о продукции в табличном интерфейсе (рис.21)

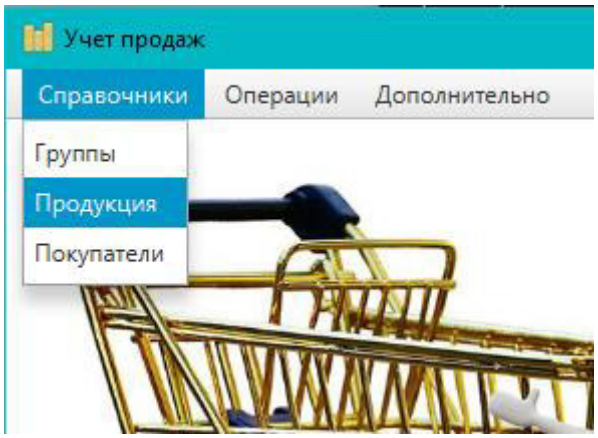
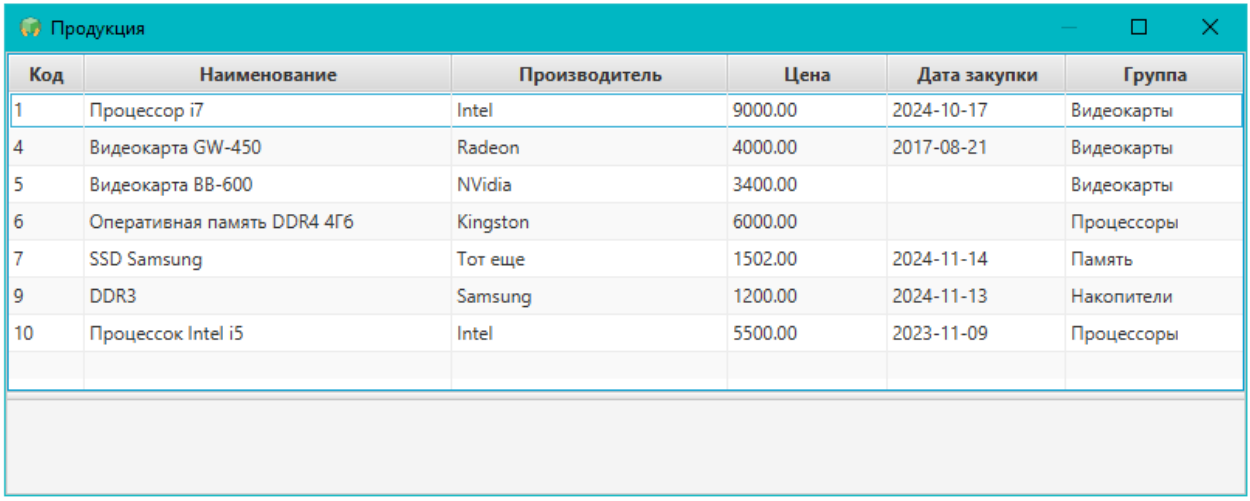


*Рис.20 Назначение пункту меню обработчика*




| Код | Наименование | Производитель | Цена | Дата закупки | Группа |
|---|---|---|---|---|---|
| 1 | Процессор i7 | Intel | 9000.00 | 2024-10-17 | Видеокарты |
| 4 | Видеокарта GW-450 | Radeon | 4000.00 | 2017-08-21 | Видеокарты |
| 5 | Видеокарта BB-600 | NVidia | 3400.00 | | Видеокарты |
| 6 | Оперативная память DDR4 4Г6 | Kingston | 6000.00 | | Процессоры |
| 7 | SSD Samsung | Тот еще | 1502.00 | 2024-11-14 | Память |
| 9 | DDR3 | Samsung | 1200.00 | 2024-11-13 | Накопители |
| 10 | Процессок Intel i5 | Intel | 5500.00 | 2023-11-09 | Процессоры |

*Рис.21 Вывод продукции в табличном интерфейсе*

Отметим, что значения в колонке **Цена** выровнены по левой границе. Более правильным является вывод с выравниванием по правой границе, чтобы в разных строках разряды чисел одинакового веса выводились друг под другом. Для достижения этого применим к значениям данной колонки правило стиля CSS «выравнивание по правой границе»:

«-**fx-alignment:center-right;**»

Создадим это правило в SceneBuilder (рис.22).

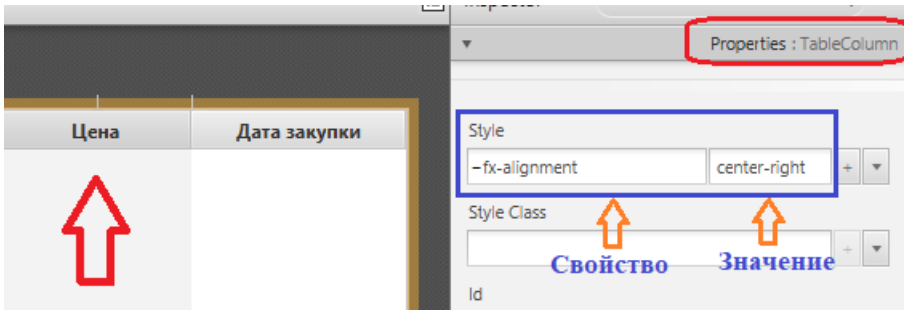



*Рис.22 Установка «выравнивания» по правой границе*

Также изменим формат вывода даты в привычный ДД-ММ-ГГГ. Ячейка даты LocalDate получает значение в команде:

```
indateColumn.setCellValueFactory(cellData ->
                cellData.getValue().indateProperty());
```

Получив значение ячейки, выполним ее форматирование, добавив код:

```
DateTimeFormatter formatter = DateTimeFormatter.ofPattern
                                        ("dd-MM-uuuu");
indateColumn.setCellFactory(cellData -> {
```

```
    return new TableCell<Product, LocalDate>() {
        @Override
        protected void updateItem(LocalDate item, boolean empty) {
            super.updateItem(item, empty);
            if (item == null || empty) {
              // Если дата не заполнена
                setText(null);
            } else {
                // Форматирование даты
                setText(formatter.format(item));
                  }
              }
          };
});
```

Также добавим выравнивание даты по правой границе.

После внесенных изменений вывод таблицы примет вид, представленный на рисунке 23.

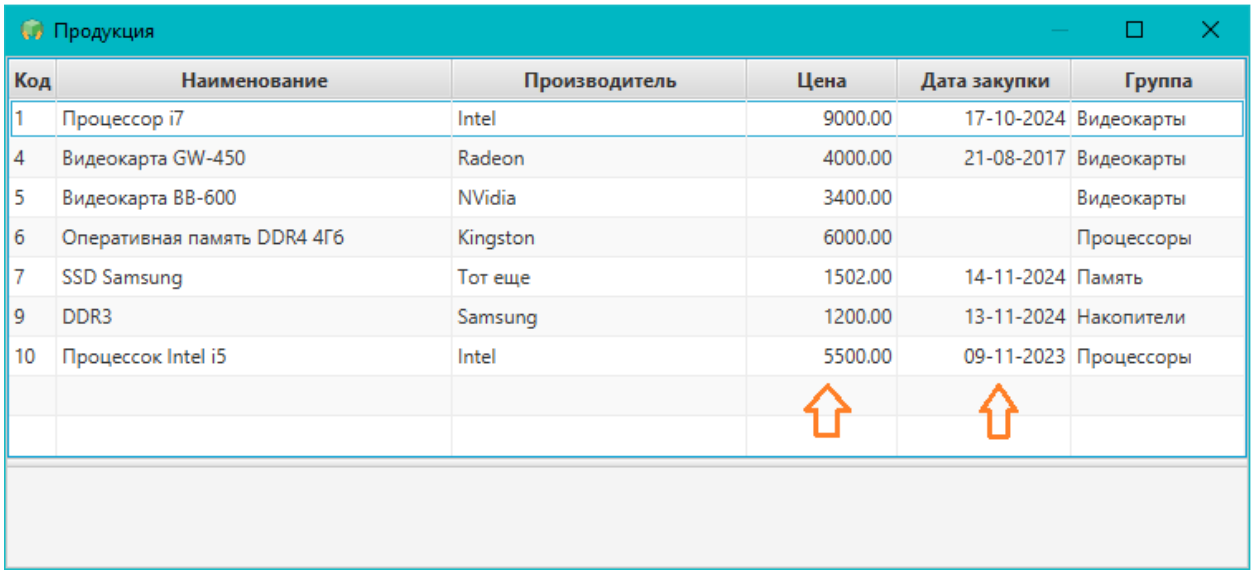



***Рис.23 Вывод продукции в отформатированном виде***

## 2. Задание

1) Изучить теоретические сведения
2) Создать БД по индивидуальным вариантам лабораторной работы 7 и заполнить ее данными
3) Дополнить приложение, созданное в лабораторной работе 7, формами вывода данных справочников в виде таблицы

## 3. Содержание отчета

1) Схема физической модели данных
2) Класс данных для одного справочника
3) Текст ОДНОГО метода получения данных из класса DBManager
4) Текст ОДНОГО класса-контроллера и файла описания интерфейса FXML
5) Метод основного окна для вызова формы Просмотра справочника
6) Скриншот формы Просмотра справочника с данными